Programming Guidelines

Table of Contents

1. Development Environment
2. Code Organization
3. Qt-Specific Guidelines
4. Object-Oriented Design Principles
5. Error Handling
6. Testing Standards
7. Documentation Requirements
8. Version Control
9. Build and Deployment

Development Environment

1.1 Required Softwares

Tool	Version	Purpose	Installation Link
C++ Compiler	Clang 14+	Performance & Optimization, Error Reporting, Less Memory Usage	https://github.com/llvm/llvm-project/releases
Qt Framework	Qt 6.10	Most Powerful C++ GUI Frame work, Cross-platform, Qt Creator IDE, Built in Classes	https://www.qt.io/development/qt-educational-license#application
CMake	CMake 4.2.3	Cross-platform, Generates Makefiles, flexible configuration, manages complexity, user friendly	https://cmake.org/download/
Git	2.30+	Version Control, familiarity	https://git-scm.com/
Qt Creator	N/A	GUI IDE	Included with Qt
VScode	N/A	Common IDE for C++ programming	https://code.visualstudio.com/download
Doxygen	1.16.1	Document Generator	https://www.doxygen.nl/download.html

1.2 Development Environment Setup

1. Clone Repository

Git clone	https://github.com/WillScoot-crtl/AEEDevelopment.git
Cd	aeedevelopment

2. Install Qt 6

> Download Qt from URL

> Apply for a Qt Educational License

> Qt Edu for Developers [Get Qt now]

> Join Qt Academy

> Install Qt

Qt Framework will be used as an extension from visual studio code using the following commands:

> Qt: Register Qt Installation

3. Configure Build Environment

Bash

Build Environment	Build Project	Run Test
mkdir build	Cmake --build	ctest
cd build		
cmake . .		

1.3 IDE Configuration

Qt	VScode
CMakeLists.txt	
6.10	

Code Organization

2.1 Directory Structure

- aeedevelopment/
  - CMakeLists.txt

- README.md
- .gitignore
- docs/
    - requirements/
    - design/
    - user-manual/
- src/
    - main.cpp
    - CMakeLists.txt
    - Core/
        - tokenizer.h
        - tokenizer.cpp
        - parser.h
        - parser.cpp
        - evaluator.h
        - expressiontree.h
        - expressiontree.cpp
    - gui/
        - mainwindow.h
        - mainwindow.cpp
        - mainwindow.ui
        - parsetreewidget.h
        - parsetreewidget.cpp
        - resources/
            - resources.qrc
    - Utils/
        - errorhandler.h
        - errorhandler.cpp
    - CMakeLists.txt
- Tests/
    - test_tokenizer.cpp
    - test_parser.cpp
    - test_evaluator.cpp
    - CMakeLists.txt
- resources/
    - icons/
    - images/

2.2 File Naming Conventions

Header and Implementation files will remain as a lowercase single line.

C++ Source Files

expressiontree.h

expressiontree.cpp

Qt UI Files:

mainwindow.h

mainwindow.cpp

Test Files:

Testing files will only be implemented as test_[component_name].cpp

test_tokenizer.cpp

Documentation Files:

Files will either be upper case or title case

README.md

2.3 Header and Implementation File Structure

Header File Template (.h)

```
#ifndef TOKENIZER_H

#define TOKENIZER_H


#include <QString>

#include <vector>


class Token;

Class Tokenizer

{

Public:

Tokenizer();

std::vector<Token> tokenize(const QString& expression);

Private:
```

```
};

#endif
```

Implementation File Template (.cpp)

```
#include "tokenizer.h"

#include [other headers]

Tokenizer::Tokenizer()

{

}

Std::vector<Token> Tokenizer::tokenize(const QString& expression)

{

{
```

Qt-Specific Guidelines

3.1 Qt Coding Style Alignement

This project will follows conventional Qt coding styles

3.2 Signal and Slot Usage

Connecting signals to slots will use the new-style

```
connect(button, &QPushButton::clicked, this, &MainWindow::onEvaluateClicked);
```

3.3 Qt Code

All UI will be written in C++ code

3.4 Qt Types vs Standard Library

Purpose	Qt Type	STL Type	Decision
String	QString	std::string	Q
Dynamic Array	QVector<T>	std::vector<T>	STL
List	QList<T>	std::list<T>	STL
Map	QMap<K,V>	std::map<K,V>	STL

3.5 Memory Management in Qt

Qt parent-child for automatic memory management will have the child widget deleted when parent is deleted

```
QPushButton* button = new QPushButton("Evaluate", this);
```

Smart Pointer Usaged

Memory management is primarily handled by QObject parenting systems for QObject derived classes. Standard C++ spoint pointers (std::unique_prt and std::share_ptr) will be conventional for non-QObject types.

3.6 Qt Resource System

```
<RCC>
    <qresource prefix="/icons">
        <file>calculator.png</file>
        <file>tree.png</file>
    </qresource>
    <qresource prefix="/styles">
        <file>darkmode.qss</file>
    </qresource>
</RCC>
```

Acessing Resources:

```
QIcon icon(":/icons/<file>");
```

Object-Oriented Design Principles

4. 1Engine Classes

Class	Responsibility
Token	Represents a ingle token(number, operator, parenthesis)
Tokenizer	Converts input to token stream

ExpressionNode	Node in the parse tree
Parser	Builds parse tree/AST from token
Evaluator	Traverses parse tree and computes result
ErrorHandler	Manages error message

4.2 GUI Classes

Class	**Responsibility**
MainWindow	Main application window
ParseTreeWidget	Custom widget to draw the tree
EvaluationStepsWidget	Shows step-by-step computation
HistoryPanel	Displays expression history
ExpressionInput	Custom input field

Error Handling

5.1 Error Handling Strategy

Error handling will be done through C++ expection. Exepctions are a preferred mechanism in modern C++ for reporting errors that prevent afunction from completing an assigned task. The following format will serve as our exception hierarchy

Class ExpressionException : Public std::expection {

};

Class TokenizerException : Public ExpressionException {

};

Class EvaluationException : public ExpressionException{

};

5.2 Error Reporting to User

```
QMessageBox::critical(this, "Evaluation Error",
                      "Invalid expression: unmatched parentheses at position 5");
```

5.3 Logging and Debugging

```
qDebug() << "Tokenizer: processing expression:" expression;

qWarning() << "Parser: Unexpected token at position" << pos;

QCritical() << "Evaluator: division by zero";
```

Testing Standards

6.1 Testing Framework

Frame work: Qt Test

- Documentation: https://doc.qt.io/qt-6/qttest-index.html

Test organization:

Test/

- Test_tokenizer.cpp # tests for tokenizer class
- Test_parser.cpp # tests for parser class
- Test_evaluator.cpp # tests for evaluator class
- Test_expressiontree.cpp #tests for expressiontree class
- CMakeLists.txt

6.2 Test Structure

Testing method templates

```
Void TestEvaluator::testSimpleAddition()

{

        // Arrange

        Evaluator evaluator;

        QString expression =  "2 + 3"

        // Act

        Double result = evaluator.evaluate(expression);



        //Assert
```

```
    QCOMPARE(result, 5.0);
}
```

Test with QCOMPARE, QVERIFY:

```
//Exact comparison
QCOMPARE(actualy, expected);
//boolean check
QVERIFY(result > 0);
// Exception testing
QVERIFY_EXCEPTION_THROWN(evaluator.evaluate("5/0, EvaluationException);
```

Documentation Requirements

7.1 Code Documentation

Code will only be document using Doxygen. Requied documentation will include all public-classes and method

Example:

```
/**
 • @class Parser
 • @brief Parses tokenized arithmetic expressions into an expression tree
 •
 • The Parser class takes a sequence of tokens from the Tokenizer and
 • constructs an abstract syntax tree representing the expression structure.
 • It enforces operator precedence and handles parentheses grouping.
 •
 • Example usage:
 • @code
 • Parser parser;
 • std::vector tokens = tokenizer.tokenize("2 + 3 * 4");
 • ExpressionTree* tree = parser.parse(tokens);
   @endcode
*/
```

Version Control

8.1 Repository Setup

Repository: https://github.com/WillScoot-crtl/AEEDevelopment.git

Acess: Team members will be added as contributors to the GitHub repositry

8.2 Branching Strategy

Branch Types:

- main branch: stabe, release code

- develop branch: integration

- feature/* branches: individual feature development

-bugfix/* branches: bug fixes

8.3 Commit Guidelines

Commit Message Format will remain in a conventional format

- feat: new feature
- fix: bug fix
- docs: documentation changes
- style: code style/formatting
-

8.4 Pull Request / Merge Request Process

PR Description Template

Description

- what does this PR do

Type of Change

- Bug fix
- New feature
- Breaking change
- Documentation update

Checklist

- Code follows style guidelines
- Comments added
- Tests added
- Documentation updated
- All tests pass

Build and Deployment

9.1 Build and Deployment

Build system – CMake

CMakeLists.txt Structure

```
CMake_minimum_required(VERSION 3.16)

Project(aeedevelopment VERSION 1.0)


Set(CMAKE_C20_STANDARD)

Set(CMAKE_C20_STANDARD_REQUIRED ON)


#Qt setup

Set(CMAKE_AUTOMOC ON)

Set(CMAKE_AUTORCC ON)

Set(CMAKE_AUTORCC ON)

Find_package(Qt6 REQUIRED COMPONENTS Widget core GUI test)


Add_subdirectory(src)

Add_subdirectory(tests)
```

9.2 Build Configurations

Debug Build

```
Cmake –DCMAKE_BUILD_TYPE=Debug . .

Cmake –build
```

Release Build

```
Cmake –DCMAKE_BUILD_TYPE=Release . .
```

Cmake –build

9.3 Continuous Integration

CI Service: Github Action

CI Pipeline:

1. Checkout code

2. Install dependencies

- Qt | 6.10 | GUI Framework

3. Build Project

4. Run tests

5. Generate reports

9.4 Application Packaging

Windows:

- Windeployqt

MacOS

- .app bundle
- Using macdeployqt
- DMG creation

Linux

- .deb